Module 6 — Applications of AI and Current State of Research

NLP • Expert Systems • Prompt Engineering • GitHub Copilot • RAG • Biomedical & Life Sciences

1. Natural Language Processing (NLP)

1.1 Definition & Scope

Natural Language Processing (NLP) is the sub-field of Artificial Intelligence concerned with enabling computers to understand, interpret, manipulate and generate human language — both written and spoken — in a meaningful and useful way. It is the intersection of Computer Science, Artificial Intelligence and Human Language (Linguistics).

NLP draws on three foundational areas:

- Linguistics — grammar, morphology, syntax, semantics, pragmatics.
- Computer Science — algorithms, data structures, ML and deep-learning models.
- Statistics & Machine Learning — probabilistic models, neural networks, Transformers.

Input/Output of an NLP system can be Speech or Written Text.

Key Goals of NLP

- Machine Translation — translating between human languages.
- Information Extraction — pulling structured facts from unstructured text.
- Question Answering — finding precise answers from natural-language queries.
- Sentiment Analysis — detecting opinion and emotion in text.
- Summarisation — condensing long documents automatically.
- Dialogue Systems — conversational agents and chatbots.

1.2 Components of NLP

(a) Natural Language Understanding (NLU)

Helps the machine understand and analyse human language by extracting metadata from content such as concepts, entities, keywords, emotion, relations and semantic roles. Used in business applications to understand the customer's problem in both spoken and written language.

NLU involves:

- Mapping the given input into a useful representation.
- Analysing different aspects of the language.

(b) Natural Language Generation (NLG)

Acts as a translator that converts computerised data into natural-language representation. It mainly involves Text Planning, Sentence Planning, and Text Realisation.

1.3 Steps in NLP — The 10-Stage Pipeline

Errors at any stage propagate downstream, so each stage must be tuned for the application.

1. Text Acquisition — Collect raw input (web scraping, PDF extraction, speech-to-text, database reads). Handle encoding (UTF-8, Unicode) and language detection.
2. Text Cleaning — Remove HTML tags, boilerplate, special characters and duplicate whitespace; normalise Unicode; fix encoding artefacts; handle contractions.
3. Sentence Segmentation — Split cleaned text into individual sentences using punctuation rules, abbreviation lists and ML boundary detectors (e.g. spaCy sentencizer). Paragraph → Sentences.
4. Tokenisation — Break sentences into words or sub-word tokens. Strategies: whitespace, rule-based, BPE (GPT), WordPiece (BERT), SentencePiece (T5). Sentences → Tokens.
5. Morphological Analysis (Stemming & Lemmatisation) — Stemming (Porter, Snowball) and lemmatisation (WordNet) reduce vocabulary and normalise word forms; e.g., intelligence / intelligent / intelligently → root 'intelligen'.
6. Stop-Word Removal — Remove low-information words like is, and, a, the.
7. POS Tagging — Label each token with its grammatical role: NN (noun), VBZ (verb), JJ (adjective), DT (determiner), IN (preposition).
8. Named Entity Recognition (NER) — Identify and classify named spans: PERSON, ORG, GPE, DATE, MONEY, PRODUCT (tools: spaCy, Stanford NER, BERT-NER). Example: 'Steve Jobs introduced iPhones'.
9. Syntactic Parsing / Dependency Parsing & Chunking — Build constituency tree (CFG / CYK / Earley) or dependency graph (arc-eager, neural). Chunking groups individual tokens into bigger phrases.
10. Semantic Analysis & Pragmatic Processing — Word Sense Disambiguation, Semantic Role Labelling (FrameNet, PropBank), coreference resolution, FOL / lambda-calculus representation, followed by discourse coherence, speech acts, Gricean maxims and implicature.

Worked Example — Input: ' John\'s dog jumped quickly over the lazy foxes. '

- Text Acquisition: detect encoding = UTF-8, language = English.
- Text Cleaning: strip … → "John's dog jumped quickly over the lazy foxes."
- Sentence Segmentation: 1 sentence detected.
- Tokenisation (BPE): ["John","'s","dog","jumped","quickly","over","the","lazy","foxes",".“]

- Morphological Analysis: jumped→jump | quickly→quick | foxes→fox | John's→John.
- POS Tagging: John/NNP 's/POS dog/NN jumped/VBD quickly/RB over/IN the/DT lazy/JJ foxes/NNS ./.
- NER: PERSON: 'John' (B-PER); all other tokens: O.
- Syntactic Parse: [S [NP John's dog] [VP jumped quickly [PP over [NP the lazy foxes]]]].
- Semantic Analysis: Agent="John's dog", Predicate="jumped", Manner="quickly", Location="over the lazy foxes".
- Pragmatic Processing: Speech act = Declarative; no anaphora; no implicature; sentiment = Neutral.

1.4 Parsing and Semantic Interpretation

Parsing is the syntactic backbone of NLP — it converts a flat string of tokens into a hierarchical structure that reveals grammatical relationships.

Constituency Parsing

- Builds a phrase-structure tree from a Context-Free Grammar (CFG).
- Operationalised by CYK and Earley algorithms.
- PCFG (Probabilistic CFG) adds probability scores for disambiguation when a sentence has multiple valid parse trees.

Dependency Parsing

- Builds a directed graph of head-dependent relations between words.
- Algorithms: arc-eager transition-based parser, neural graph-based parsers.
- Captures who-did-what-to-whom relations directly.

Semantic Interpretation (Stage 9)

- Word Sense Disambiguation (WSD) — pick the correct sense of a polysemous word from context.
- Semantic Role Labelling (SRL) — identify Agent, Patient, Instrument, Location etc. using FrameNet / PropBank.
- Coreference resolution — link pronouns and nominals back to their antecedents.
- Formal meaning representation — First-Order Logic (FOL) or lambda-calculus expressions.

1.5 Discourse and Pragmatic Processing

Pragmatic processing (Stage 10) is the deepest layer of NLP — it considers meaning beyond the sentence, in context.

- Discourse coherence — modelled by Rhetorical Structure Theory (RST); how sentences hang together.

- Speech acts — identifying whether an utterance is a statement, question, command, promise, etc.
- Gricean maxims — quantity, quality, relation, manner; assumptions about cooperative communication.
- Presupposition — what the speaker assumes the hearer already knows.
- Implicature — what is implied but not stated (e.g., 'Some students passed' implies not all).
- Anaphora resolution — resolving 'he', 'it', 'they' to the right entity.
- Sentiment in context — sarcasm, irony, polarity shifts.

1.6 Implementation Aspects of Syntactic Analysis

- Grammar formalism — choose CFG, PCFG, dependency grammar, or combinatory categorial grammar (CCG).
- Algorithmic engine — CYK runs in $O(n^3 \cdot |G|)$; Earley handles arbitrary CFGs; shift-reduce / arc-eager run in linear time.
- Disambiguation strategy — PCFG, lexicalised PCFG, or neural scoring to pick the best parse from many.
- Lexical resources — POS tagger, treebanks (Penn Treebank, Universal Dependencies), morphological analyser.
- Neural parsers — BiLSTM and Transformer-based dependency parsers (e.g., Stanford Stanza, spaCy v3 transformer pipeline).
- Engineering trade-offs — accuracy vs. latency, full parse vs. shallow chunking, language-specific tokenisation for morphologically rich languages.

1.7 Core Challenges in NLP

- Ambiguity — lexical ('bank'), syntactic ('I saw the man with the telescope'), semantic ('He fed the dog biscuits').
- World knowledge & common sense — Winograd Schema problems; models lack grounded physical and social common sense.
- Low-resource languages — 7,000+ languages exist but annotated data exists for only ~100; most SOTA models are English-centric.
- Long-range dependencies — center-embedded sentences and quadratic attention cost in Transformers.
- Sarcasm, irony & figurative language — require pragmatic and cultural reasoning beyond surface syntax.
- Bias & fairness — gender, racial and cultural stereotypes propagate from biased training corpora.

1.8 Strengths of Modern NLP Systems

- Scalability — process millions of documents at low marginal cost.

- Multilingual capability — mBERT, XLM-R cover 100+ languages; zero-shot cross-lingual transfer.
- Contextual understanding — Transformers (BERT, GPT-4) capture long-range, polysemous meaning.
- Transfer learning — pre-train once, fine-tune cheaply; few-shot capability.
- Broad task coverage — translation, QA, summarisation, NER, generation from one backbone.
- State-of-the-art performance — match or exceed humans on SuperGLUE, SQuAD, WMT benchmarks.

1.9 Weaknesses & Limitations of NLP

- Hallucination & factual errors — LLMs confidently invent facts without retrieval grounding.
- Lack of true understanding — statistical pattern matching, not causal reasoning (Clever Hans effect).
- High computational cost — training a GPT-4-scale model costs millions of dollars.
- Catastrophic forgetting — fine-tuning degrades prior-task performance.
- Interpretability — attention is not a reliable explanation; a barrier in regulated domains.
- Data hunger & privacy — large corpora include personal data; GDPR and memorisation risks.

1.10 Applications of NLP

- Virtual assistants — Siri, Alexa, Google Assistant (intent + slot filling + dialogue).
- Machine translation — Google Translate, DeepL, AWS Translate.
- Clinical NLP — ICD coding, drug-interaction detection, radiology reports.
- E-commerce & sentiment — aspect-level review analysis, fake-review detection.
- Education & grading — automated essay scoring, intelligent tutoring (Duolingo).
- Information retrieval — BERT re-ranks billions of search results daily.
- Code intelligence — GitHub Copilot, Code Llama, Cursor.
- Legal & financial NLP — contract analysis (Harvey AI), earnings-call sentiment.
- Speech & conversational AI — Whisper ASR, speaker diarisation, call-centre automation.
- Other — Question Answering, Spam Detection, Sentiment Analysis (opinion mining), Spelling Correction, ChatBots.

2. Expert Systems

2.1 Introduction & Definition

An Expert System (ES) is an artificially intelligent program that implements the knowledge and reasoning processes of a human expert in a certain field. Expert systems are computer applications developed to solve complex problems in a particular domain, at the level of extraordinary human intelligence and expertise. They are highly knowledge-based, interactive and reliable.

Dr. Edward Feigenbaum — often called the 'father of expert systems' — defines an ES as: "An intelligent computer program that employs knowledge and inference procedures to solve problems that are considered difficult enough to require significant human expertise for their solutions."

ES was introduced by researchers at the Stanford University Computer Science Department and is one of the prominent research domains of AI.

2.2 Components of an Expert System

An ES typically has five tightly-coupled components: User Interface, Knowledge Base, Inference Engine, Explanation Subsystem, and Knowledge Base Editor.

(a) User Interface

The simplest ES is a question-and-answer system; the most complex has a full GUI. Users provide input and receive output through this interface.

- Input — speech (microphone), forms, questionnaires, dialogue boxes, sensors, command prompts.
- Output — speech via speakers, graphs, modem actions, graphical output, textual results.

(b) Inference Engine — the 'brain' of the ES

The inference engine contains customised algorithms, rules and reasoning methods. It interacts with the Knowledge Base to retrieve background and current information, rules and facts. Algorithm families typically used:

11. Forward / backward chaining with if-then-else rules.
12. Machine-learning algorithms.
13. Neural networks.
14. Bayesian networks.
15. Decision trees.
16. Customised algorithms.

(c) Knowledge Base

A collection of facts and rules — voluminous, hard to characterise, and constantly changing. Because of this, an ES targets one specific problem domain. The KB is typically split into:

- Domain-specific knowledge — general expertise about the field.
- Problem-specific knowledge — facts about the current case being solved.

(d) Explanation Subsystem

Unique to ES. When asked, the ES can explain its reasoning — from the input through every intermediate state to the conclusion. Useful for debugging (identifying missing facts/rules) and for end-user trust when there is a difference of opinion.

(e) Knowledge Base Editor

Allows subject-matter experts to add, modify and convert rules in the knowledge base, keeping the ES flexible and open to adopting new knowledge.

2.3 Expert Systems vs. Traditional Systems

Aspect	Traditional System	Expert System
Approach	Algorithmic / procedural	Knowledge-based, heuristic, rule-driven
Knowledge & Data	Data and program are interleaved	Knowledge base separate from inference engine
Reasoning	Numerical computation	Symbolic inference (forward / backward chaining)
Handling Uncertainty	Limited; usually deterministic	Built-in via fuzzy logic, Bayesian, certainty factors
Explanation	No explanation of how output was produced	Explanation subsystem traces the reasoning
Modification	Code must be rewritten	Update rules in the KB without changing code
Output	Single correct answer	Advice, recommendation, possibly with confidence
Goal	Process data efficiently	Mimic decision-making of a human expert

ES also differ from neural networks: ES combines domain-specific and case-specific knowledge through symbolic inference, while neural networks use layered non-linear neurons trained on previous patterns. Example: an ES for crime analysis identifies suspect/culprit/motive; a neural network identifies crime patterns and growth.

2.4 Characteristics of Expert Systems

- High Performance — solves complex problems of a specific domain with high efficiency and accuracy.
- Understandable — accepts input in human language and produces output in the same way.
- Reliable — generates accurate, error-free output.
- Highly Responsive — answers complex queries in a very short time.
- Flexible — the knowledge-base editor lets it adopt new knowledge.
- Handles Complexity — built to tackle challenging decision-making problems.
- Performance depends on the quantity and quality of knowledge integrated in the ES.

2.5 Roles in ES Implementation

(a) Domain Expert

One or more people with hefty proficiency in the target domain. Need not have AI / programming knowledge — their job is to supply the knowledge.

(b) Knowledge Engineer

Specialist in AI / computer science who designs, programs and tests the ES. Usually multiple knowledge engineers per project. A knowledge engineer must be acquainted with:

17. Various knowledge-representation techniques.
18. Intelligent searching strategies.
19. Expert-system tools.
20. Software-engineering methodologies that formalise and accelerate development.
21. Other expert systems in similar domains — to adopt best practices and avoid past mistakes.

(c) User

End user of the ES, technical or non-technical, possibly assuming different roles. Example: in MYCIN, the physician entering symptoms is one type of user; the person trying to understand the diagnosis and prescription is another.

2.6 ES Implementation Process

Phase 1 — Problem & Goal Identification

Analogous to requirements analysis. Identifies external requirements, I/O forms, problem environment and the user. Knowledge engineer gathers information from books, manuals, training guides and the domain expert. Iterative refinement continues until the problem description is accepted. Output: finalised objectives, technical team, manuals, cost and time-frame.

Key questions to answer during identification:

- Complexity class of the problem?
- How to characterise / define the problem?
- Key subproblems and partitions of the subtasks?
- The data and data sources?
- Taxonomy and key terms and their associations?
- Expected solution?
- Human skill qualities important in problem solving?
- Role, characteristics and volume of knowledge required?
- Tricky and complex circumstances needing special treatment?
- Uncertainties involved in the problem and solution domains?

Phase 2 — Conceptualisation

Primary phase of knowledge acquisition. The knowledge engineer divides the problem into subproblems and represents relationships between objects/processes diagrammatically; the domain expert validates and refines until both are on the same page. Key questions:

- Availability of various types of data?
- What goes into the knowledge base, what is assumed, what is inferred?
- Relationships of objects in the problem domain?
- Problem solution processes and their constraints?
- Data flow in the system?
- Nature of the human-expert decisions and outcomes?
- Resources and inputs needed to reach a decision?
- Elements introducing uncertainty and how they are treated?
- Expected consistency between ES output and the human expert's recommendation?

Phase 3 — Formalisation (Design)

Connects the problem with the appropriate AI technologies. The knowledge engineer chooses the best technical approach (possibly combining technologies based on the nature of the subproblems) and encodes the domain-specific rules. The domain expert reviews and the cycle repeats. This is usually the most time-consuming phase. Output: formal specification and approved encoded knowledge base.

Phase 4 — Implementation

The formalisation is implemented as the first prototype. The first version is often a throw-away — the team learns which technologies work and which don't. Re-formalisation may follow.

Phase 5 — Testing

Confirms real expertise of the developed ES. Because knowledge is voluminous, hard to characterise and constantly changing, the prototype is rarely complete. Testing exposes missing

knowledge, formulation errors and design/implementation weaknesses. The team iterates back to earlier phases. After the system works on simple problems, it is tested on complex situations. The ES passes only when it performs at par with a human expert on real-life problems.

2.7 Applications of Expert Systems

22. Helping investors with financial investments and risk management.
23. Medical diagnosis — especially in remote-medicine facilities.
24. Exploration of oil fields in the oil industry.
25. Maintenance and repair of equipment in manufacturing.
26. Management of complex production processes.
27. Help-desk management.
28. Malicious-software detection.
29. Anomaly detection.
30. Cargo and airline scheduling.
31. Stock-market analysis.
32. Profile analysis for loan approvals, etc.

Classic Examples

- MYCIN (1960s) — earliest medical-diagnosis ES; used >600 inference rules with backward chaining; could also prescribe drugs.
- DENDRAL (1960s) — identified unknown organic molecules using graph-theory algorithms. Many ES, including MYCIN, were derived from it.

Case Study 1 — Web-Based Fuzzy Expert System for Lung Cancer Diagnosis

- User Interface: web form; login with Medical Record Number; output is diagnosis + treatment suggestion.
- Knowledge Base: MySQL DB with variables — anamnesis, doubling, tumour time, degree of smoking, age, performance status; 243 rule combinations gathered from doctor interviews.
- Inference Engine: fuzzy-logic-based; inputs classified into fuzzy sets, weighted average computed, de-fuzzification → diagnosis (absent / benign / malignant).
- Features: diagnoses lung cancer using fuzzy rules and supporting data; gives diagnosis plus treatment suggestion.

Case Study 2 — Expert System for Financial Decision Making

- User Interface: Windows-based GUI with menus and graphics; user can query, input information, and receive advice in their language.
- Knowledge Base Editor: lets the subject-matter expert compose rules and convert between forms.
- Knowledge Base: facts + heuristic rules; Prolog is widely used. Examples:
- falls(investment) :- rises(interest_rate).

- rises(investment) :- falls(interest_rate).
- rises(interest_rate) :- rises(stock_price).
- falls(interest_rate) :- falls(stock_price).
- Inference Engine: rules with if-clause and then-clause; uses forward and backward chaining for reasoning.

2.8 Advantages of Expert Systems

33. Schooling human experts is expensive; ES is comparatively easy to build, accepts input from multiple experts, can be copied / relocated / put to work for long hours.
34. High availability — never needs vacation or sick leave.
35. Accumulated knowledge of multiple human experts improves decision quality.
36. Cheaper one-time investment vs. paying human experts per session.
37. Fast, accurate, fewer human errors when implemented properly.
38. Free from human emotions, tensions, anxiety or fatigue.
39. Holds vast amounts of information and never forgets the tiniest detail.
40. Outlives human experts; can be modified as knowledge and technology evolve.
41. Consistent — does not act unpredictably like humans.
42. Other benefits: Availability, Less Production Cost, Speed, Less Error Rate, Reducing Risk (works in dangerous environments), Steady Response (not emotional / tense / fatigued).

2.9 Limitations of Expert Systems

- Cannot give reliable responses in unforeseen situations.
- Knowledge is vast; missing facts/rules or uncertainty cause incorrect decisions.
- Building and maintenance are very expensive — large systems require significant development time and computer resources.
- Cannot be creative; tends to give the same answer in similar situations.
- Works only in its specific domain — fails outside scope.
- Cannot be flexible or adopt new learning unless explicitly programmed.
- Initial setup cost and time are higher than using a human expert.
- Limitations of the underlying technology and the difficulty of knowledge acquisition.

3. Case Studies Based on Current Research in AI

3.1 Key Domains of Current AI Research

- Healthcare — Disease prediction, medical imaging diagnosis.
- Finance — Fraud detection, credit-risk scoring.
- Autonomous Vehicles — Self-driving perception, planning and control.
- AI Chatbots and Assistants — Conversational AI, customer service automation.

3.2 Case Study — AI in Healthcare

- Detects cancer from medical images (mammograms, CT scans, histopathology slides).
- Uses deep-learning models — CNNs, Vision Transformers, U-Net for segmentation.
- Improves diagnostic accuracy beyond average radiologist performance on specific tasks.
- Several FDA-cleared AI products are deployed in clinical decision support.

3.3 Other Notable Case Studies

- Finance — fraud-detection systems flagging anomalous transactions in real time using gradient-boosted trees and graph neural networks.
- Autonomous Vehicles — Tesla Autopilot, Waymo, Cruise: combine LiDAR, vision, and reinforcement learning for path planning.
- AI Chatbots and Assistants — ChatGPT, Claude, Gemini, Copilot for customer support, productivity and research.

4. Prompt Engineering

4.1 What is Prompt Engineering?

Prompt engineering is the practice of designing effective inputs (prompts) for AI / LLM models in order to obtain high-quality, reliable outputs. It improves output quality and is heavily used with systems like ChatGPT, Claude, and GitHub Copilot.

4.2 Types of Prompts

- Zero-Shot Prompts — Ask the model to perform a task with no examples (e.g., 'Translate this sentence to French: …').
- Few-Shot Prompts — Provide a few input/output examples in the prompt so the model can imitate the pattern.
- Chain-of-Thought (CoT) Prompts — Ask the model to 'think step by step', exposing intermediate reasoning to improve accuracy on math and logic.
- Instruction-Based Prompts — Give an explicit instruction or role (e.g., 'You are a senior data scientist. Summarise this paper in 5 bullet points.').

4.3 Prompt Failures

- Ambiguous prompts — Vague wording leads to unfocused answers.
- Hallucinations — Model produces confident but factually wrong responses.
- Incomplete outputs — Truncation, missing steps, or skipped requirements.
- Misinterpretation — Model latches on to the wrong intent of the request.

4.4 Bias in Prompts

- Training-data bias — The model amplifies biases present in its training corpus.
- Cultural bias — Western-centric outputs; under-representation of other cultures.
- Gender bias — Stereotyped associations (e.g., engineer = he, nurse = she).
- Can lead to unfair or harmful results in hiring, lending, or moderation use cases.

4.5 Applications of Prompt Engineering

- Content generation — articles, marketing copy, emails.
- Code generation — completion, refactoring, documentation.
- Customer support — automated reply drafting and triage.
- Automation tasks — workflow orchestration, data extraction, summarisation.

5. AI Copilot — GitHub Copilot

5.1 What is GitHub Copilot?

- An AI coding assistant developed by GitHub and OpenAI.
- Suggests code in real-time as the developer types.
- Works inside popular IDEs — Visual Studio Code, JetBrains family, Neovim, Visual Studio.

5.2 Features of GitHub Copilot

- Auto-completion — completes lines, blocks, or whole functions.
- Code suggestions — proposes implementations from natural-language comments.
- Supports multiple languages — Python, JavaScript/TypeScript, Go, Java, C#, Ruby and many more.
- Improves productivity — reduces boilerplate and lookup time, used by over 1M developers (2024).
- Chat / agentic modes for explaining code, writing tests, fixing bugs, and reviewing PRs.

5.3 Considerations

- Generated code must be reviewed — may contain bugs, insecure patterns, or licensing issues.
- Should not be a substitute for engineering judgement; works best as a 'copilot', not autopilot.

6. Retrieval-Augmented Generation (RAG)

6.1 Motivation — Why LLMs Need External Knowledge

- Knowledge Cutoff — LLMs are trained on static datasets; cannot access events after the cutoff date.
- Hallucination — Without grounding, LLMs generate plausible-sounding but wrong answers.
- Domain Specificity — General LLMs lack proprietary / specialised knowledge (legal, medical, financial).
- Lack of Verifiability — Standalone LLM outputs cannot be traced to sources, making audit nearly impossible.

RAG directly addresses all four limitations by dynamically retrieving relevant, current, verifiable information at inference time.

6.2 Definition

RAG, introduced by Lewis et al. (2020, Facebook AI Research), is a hybrid AI framework that enhances large language models by coupling them with a retrieval mechanism over an external knowledge corpus, enabling the model to produce responses grounded in retrieved, up-to-date, and domain-specific information.

- Retriever — finds relevant chunks from the corpus (dense or sparse).
- Generator — LLM that conditions its output on the retrieved context.
- Knowledge Base — external corpus (PDF, DB, web, APIs); static or dynamic, structured or unstructured.

Pipeline: User Query → Query Encoder → Document Retriever → Context Fusion → LLM Generator → Grounded Response.

6.3 Core RAG Architecture

Indexing Pipeline (Offline)

43. Document Loading — ingest raw sources (PDFs, HTML, databases, APIs).
44. Chunking — split into fixed / semantic / sentence-level chunks.
45. Embedding — encode chunks via an embedding model into dense vectors.
46. Vector Store — index vectors in FAISS, Pinecone, Weaviate or Chroma.

Retrieval + Generation Pipeline (Online)

47. Query Encoding — embed the user query with the same embedding model.
48. Similarity Search — top-k vector search using cosine, dot-product, or MMR.
49. Context Assembly — concatenate retrieved chunks with the original query.

50. LLM Generation — generator produces a final grounded, cited response.

6.4 Types of RAG

(a) Naive RAG

- Simple retrieve-then-read pipeline.
- Fixed chunking and top-k retrieval.
- Single-pass generation without iteration.
- Best for: simple Q&A over static corpora.

(b) Advanced RAG

- Query rewriting and expansion.
- Re-ranking with cross-encoders (e.g., Cohere Rerank).
- Hybrid search: dense + sparse (BM25 + DPR).
- Sentence-window and parent-document retrieval.
- Best for: complex documents and enterprise search.

(c) Modular RAG

- Plug-and-play retriever / generator modules.
- Iterative and adaptive retrieval loops.
- FLARE — forward-looking active retrieval.
- RAG Fusion — multi-query generation + Reciprocal Rank Fusion (RRF).
- Best for: research pipelines and custom workflows.

(d) Agentic RAG

- LLM agent decides when and what to retrieve.
- Multi-step reasoning with tool use.
- Frameworks: ReAct, Reflexion, IRCOT.
- Self-correcting via feedback loops.
- Best for: autonomous assistants and complex tasks.

6.5 Key Retrieval Approaches

- Sparse Retrieval (BM25 / TF-IDF) — fast, interpretable, keyword-heavy queries; weak on paraphrase / synonyms.
- Dense Retrieval (DPR, Contriever) — captures semantic meaning; needs training, compute-intensive at scale.
- Hybrid Search (Sparse + Dense) — robust across query types; needs score normalisation (RRF often used).

- Re-ranking (Cross-Encoders) — accurate re-scoring of candidates; high latency, used as second stage.

6.6 Strengths of RAG

- Factual accuracy & grounding — conditioning on retrieved docs reduces hallucination and provides citations.
- No retraining required — refresh the vector store to update knowledge.
- Domain adaptability — same LLM serves legal, medical, financial verticals by swapping the KB.
- Transparency & explainability — retrieved passages provide an audit trail.
- Cost-effective scaling — storage scales, not training compute.
- Dynamic real-time knowledge — handles breaking news and live enterprise databases.

6.7 Weaknesses & Challenges

- Retrieval-quality bottleneck — generation is bounded by retrieval (GIGO).
- Context-window limitations — only a few chunks fit; critical info may be diluted.
- Retrieval-generation mismatch — different training objectives between retriever and generator.
- Multi-hop reasoning failure — struggles to synthesise information across documents.
- Latency overhead — embedding + vector search + LLM inference each add time.
- Embedding-model dependency — domain mismatch degrades retrieval.

6.8 Open Problems & Research Frontiers

- Faithfulness vs. creativity tension — grounding can suppress valid inferences.
- Chunk-boundary artefacts — arbitrary splits break coherence; semantic chunking research.
- Evaluation benchmarking gap — no single standard captures the full RAG pipeline.
- Adversarial robustness — corpus poisoning can systematically corrupt outputs.
- Long-context RAG — when does retrieval still help with 128K+ context windows?
- Multimodal RAG — extending to images, audio, code, tables.

6.9 RAG vs. Fine-Tuning

Criterion	RAG	Fine-Tuning
Knowledge Update	Instant — update vector store	Requires full retraining
Domain Adaptation	Via knowledge base	Deep behavioural adaptation
Factual Accuracy	High (grounded in retrieved docs)	Can hallucinate without retrieval
Training Cost	Low — no model retraining	High — GPU-intensive
Latency	Higher (retrieval overhead)	Lower (single forward pass)

Interpretability	High — sources traceable	Low — knowledge implicit
Long-tail Knowledge	Excellent — any doc in corpus	Limited by training coverage
Style / Tone Control	Limited — LLM's base style	Excellent — trainable

Note: In production, RAG + fine-tuning together often yields the best results.

6.10 Evaluating RAG Systems

- Context Precision (RAGAS) — fraction of retrieved chunks actually relevant.
- Context Recall (RAGAS) — fraction of relevant information that was retrieved.
- Faithfulness (RAGAS / TruLens) — is every claim supported by retrieved context?
- Answer Relevance (RAGAS) — does the answer address the user's question?
- NDCG@k — classic IR ranking-quality metric.
- Hallucination Rate (FactScore / SAFE) — proportion of statements not grounded in retrieved or world knowledge.

6.11 Real-World Case Studies / Examples of RAG

- Chatbots over PDFs — internal Q&A over policy / manual documents.
- Search engines — Bing Copilot, Perplexity, Google AI Overviews use RAG to ground answers in live web results.
- Customer support systems — answer questions over product docs and FAQs.
- Enterprise AI assistants — Microsoft Copilot for Microsoft 365 over SharePoint / OneDrive.
- Healthcare & Medicine — clinical decision support over EHR + literature; drug-interaction Q&A; medical coding; patient symptom triage.
- Legal & Compliance — contract review, case-law research, regulatory checking, due-diligence automation.
- Education & Research — Semantic Scholar / Elicit-style research Q&A, adaptive tutoring, citation-grounded writing assistants.
- Finance & Banking — earnings-call Q&A over SEC filings, risk-report summarisation, investment research, regulatory capital queries.
- Software Engineering — codebase-aware code generation (Cursor), API doc Q&A, bug triage, security-vulnerability lookup.

7. AI in Biomedical and Life Sciences

7.1 Introduction

Biomedical and life sciences have become one of the highest-impact application areas for AI. Combining medical imaging, electronic health records (EHR), omics data, scientific literature and clinical workflows, AI is moving from research labs into deployed diagnostic and therapeutic tools.

7.2 Key Application Areas

(a) Medical Imaging & Diagnostics

- Deep-learning (CNNs, Vision Transformers, U-Net) for radiology — X-ray, CT, MRI.
- Histopathology slide analysis for cancer grading.
- Diabetic retinopathy detection from fundus images (Google Health, IDx-DR).
- Several FDA-cleared AI products in clinical decision support.

(b) Clinical NLP & Electronic Health Records

- Automated ICD coding from discharge summaries.
- Drug-interaction detection from clinical notes.
- Radiology-report summarisation and structured information extraction.
- Cohort identification for clinical trials.

(c) Drug Discovery & Genomics

- AlphaFold (DeepMind) — predicts 3-D protein structure from amino-acid sequence; near-experimental accuracy.
- Generative models for molecule design — diffusion and graph neural networks.
- Genomic-variant interpretation and disease-association mining.
- Single-cell RNA-seq analysis at scale.

(d) Expert Systems for Medical Diagnosis

- MYCIN — classic ES for diagnosing bacterial infections and recommending antibiotics; >600 rules with backward chaining.
- Fuzzy ES for lung-cancer diagnosis — uses anamnesis, smoking degree, tumour-doubling time, age and performance status; classifies as absent / benign / malignant.
- ES for remote-medicine and rural triage where specialists are unavailable.

(e) RAG and LLMs in Healthcare

- Clinical decision support — RAG combines EHR + medical literature to answer point-of-care questions.

- Drug-interaction Q&A grounded in pharmacology databases.
- Medical coding assistance over ICD-10 / SNOMED.
- Patient-facing symptom triage chatbots with cited sources.

(f) Personalised Medicine & Predictive Analytics

- Risk-stratification models (sepsis early warning, readmission risk).
- Recommender systems for treatment plans based on patient phenotypes.
- Wearable-device data (continuous glucose monitors, ECG) integrated with ML for early intervention.

7.3 Benefits in Biomedical & Life Sciences

- Improves diagnostic accuracy and consistency.
- Speeds up drug discovery — months instead of years for early-stage screening.
- Provides clinical expertise in remote and under-served regions.
- Reduces clinician workload through automation of documentation and coding.
- Enables personalised, data-driven treatment.

7.4 Challenges & Ethical Considerations

- Data privacy and HIPAA / GDPR compliance for patient data.
- Bias in training data — under-representation of minorities can produce unequal outcomes.
- Interpretability — clinicians need to understand model decisions; 'black-box' models face regulatory hurdles.
- Validation and regulatory approval — FDA / CE / CDSCO clearance required for clinical use.
- Integration into clinical workflows without alarm fatigue.
- Hallucination risk for LLM-based clinical assistants — RAG and human-in-the-loop are essential.

7.5 Conclusion

AI is reshaping biomedical and life sciences by combining classical expert systems (e.g. MYCIN, fuzzy diagnostic ES), modern deep-learning (medical imaging, AlphaFold), NLP (clinical-text mining) and RAG (grounded clinical decision support). With careful attention to data quality, bias, privacy and validation, these technologies can extend the reach of expert care and accelerate scientific discovery.